Programación III: Parcial 2

Sebastián Solórzano García



Profesor: Julián Esteban Gutiérrez Posada

Universidad del Quindío
Facultad de Ingeniería
Ingeniería de Sistemas y Computación
Programación III
Armenia Quindío

INFORME DE DESARROLLO – EJERCICIO 1

Asignatura: Programación 3
Proyecto: Rutas de entrega en una red dirigida
Lenguaje: Elixir

---

1. Descripción General del Problema

El objetivo del programa es encontrar todos los itinerarios simples (sin repetir centros) entre un centro de origen y uno de destino, en una red de distribución dirigida. Cada tramo tiene un tiempo y un costo. Para cada consulta, el sistema debe:

- Filtrar tramos inválidos o duplicados.
- Identificar si el destino es alcanzable mediante un Stream infinito.
- Encontrar todos los caminos posibles usando recursión (DFS).
- Calcular los caminos óptimos según tres criterios: (a) menos tramos, (b) menor tiempo, (c) menor costo, conservando todos los empates.

---

2. Arquitectura de la Solución y Flujo de Datos

El programa se dividió en módulos independientes, respetando los requisitos de no usar Enum ni Stream en las validaciones recursivas (Requisito 7) y de usar lectura/escritura en flujo (Requisitos 1 y 4).

Módulos Implementados:

1. Tramo: Define el struct %Tramo{} y la función parse/1 para convertir texto CSV en structs validados.
2. Itinerario: Define el struct %Itinerario{} que almacena el camino, tiempo y costo total.
3. Archivos: Contiene todas las funciones de I/O por flujo (File.stream!, Stream.reject, Stream.map) y la escritura en modo append de los archivos de salida.
4. Depuracion: Función recursiva pura que recorre los tramos válidos y descarta los que van de un centro a sí mismo (A->A) y los duplicados en la misma dirección, conservando la primera aparición.
5. Grafo: Construye el mapa de adyacencia a partir de los tramos limpios y verifica si un centro existe en la red.
6. Alcance: Implementa el Stream infinito de conjuntos de centros alcanzables, deteniéndose al detectar una repetición consecutiva (estabilización).
7. Busqueda: Implementa el DFS recursivo para encontrar todos los itinerarios y calcula los tres criterios óptimos con sus empates.

8. Taller1: Orquesta el flujo principal del programa.

---

3. Desafíos Técnicos Superados

- Conservación de la línea física: Se solucionó usando Stream.with_index(1) antes de aplicar cualquier filtro de comentarios o líneas vacías.
- Validación sin Enum: La depuración y la búsqueda DFS se implementaron con pattern matching ([head | tail]), acumuladores y when guards, cumpliendo estrictamente el Requisito 7.
- Corte del Stream infinito: Se usó Enum.reduce_while/3 para consumir el Stream de alcanzables hasta que el conjunto actual fuera idéntico al anterior (estabilización).
- Salida por flujo: Se usó Enum.into(File.stream!(ruta, [:append])) para escribir líneas a medida que se generaban, sin armar strings gigantes.

INFORME DE DESARROLLO – EJERCICIO 2
Asignatura: Programación 3
Proyecto: Identificación de regiones en un mapa de colores y exportación SVG
Lenguaje: Elixir

---

1. Descripción General del Problema

El objetivo del programa es analizar un mapa representado por una cuadrícula de símbolos para encontrar regiones conectadas (horizontal y verticalmente, sin diagonales). El símbolo. representa el fondo y no forma regiones. Para cada región encontrada, el sistema debe:

- Validar la estructura del mapa y la paleta de colores.
- Encontrar las regiones usando recursión, sin visitar dos veces la misma casilla.
- Calcular el área (número de casillas), el perímetro (lados que tocan el borde o un símbolo distinto) y la caja envolvente.
- Generar un archivo CSV con el detalle, un resumen en texto con los máximos (incluyendo empates) y un archivo SVG dibujando el mapa por fragmentos.

---

2. Arquitectura de la Solución y Flujo de Datos

El programa se dividió en módulos independientes para garantizar el cumplimiento de los requisitos (lectura en flujo, escritura progresiva, validación recursiva pura sin Enum/Stream y algoritmo DFS).

Módulos Implementados:

1. Modelo: Define los structs %Color{simbolo, nombre, hex} y %Region{id, simbolo, celdas, area, perimetro, desde, hasta}.
2. Archivos: Contiene la lectura perezosa con File.stream! para mapa.txt y paleta.csv, ignorando comentarios y líneas vacías, pero conservando el índice de la línea física original.
3. Validador: Función puramente recursiva que verifica que las filas tengan igual longitud, que no existan símbolos fuera de la paleta, que el punto (.) esté declarado y que los hexadecimales cumplan el formato #RRGGBB.
4. Regiones: Implementa el algoritmo DFS (Búsqueda en Profundidad) recursivo. Una función recorre las casillas no visitadas y otra explora recursivamente los 4 vecinos de cada celda para agrupar las regiones, calculando el área, la caja envolvente y el perímetro.
5. Reportes: Calcula los máximos de área y perímetro con sus empates. Escribe regiones.csv, resumen_regiones.txt y el reporte de errores validacion_mapa.txt.
6. Svg: Genera el archivo mapa_regiones.svg dibujando los rectángulos de 40x40 px, inyectando el color correspondiente de la paleta y colocando el ID de la región centrado en su primera casilla.
7. Main: Orquesta el flujo principal, asegurando que, si hay errores de validación, no se calculen regiones ni se genere el SVG.

---

3. Desafíos Técnicos Superados

- Conservación de la línea física: Al leer archivos con File.stream!, se aplicó Stream.with_index(1) antes de rechazar líneas vacías o comentarios, permitiendo que el validador reporte la línea exacta de un error.
- Validación sin Enum/Stream: La validación estructural del mapa y la paleta se implementó exclusivamente con recursión de cola, uso de acumuladores (errores) y *pattern matching*.
- Cálculo del perímetro: Se resolvió guardando las celdas de la región en un MapSet. Para cada celda, se verifica si sus 4 vecinos (arriba, abajo, izquierda, derecha) pertenecen o no a dicho MapSet; si no pertenecen, ese lado suma al perímetro.
- Escritura progresiva del SVG: En lugar de construir un *string* gigante del documento XML, se utilizó File.open/2 junto con IO.puts/2 para inyectar el encabezado, iterar dibujando rectángulo por rectángulo, inyectar los textos y cerrar la etiqueta <svg> de forma secuencial.

Reflexión sobre el uso de Inteligencia Artificial

1. Herramientas usadas

Se empleó un asistente de IA DeepSeek como apoyo durante todo el desarrollo de ambos ejercicios. Su uso se concentró en tres fases clave:

- Diseño: Para discutir la división modular del proyecto. En el Ejercicio 1 (Rutas) ayudó a separar responsabilidades (Tramo, Depuracion, Grafo, Busqueda, Archivos). En el Ejercicio 2 (Regiones y SVG) ayudó a definir los módulos Color, Validador, Buscador, Svg y Archivos.

- Codificación: Para resolver dudas de sintaxis específicas de Elixir, como el uso avanzado de Map.update/4 para construir el grafo de adyacencia, el manejo de MapSet para evitar ciclos en la depuración, y la generación manual de formato XML (<svg>, <rect>, <text>) sin depender de librerías externas.

- Depuración: Para detectar y corregir errores lógicos. Por ejemplo, un bucle infinito en el Stream de alcanzables del Ejercicio 1, o un ajuste en la lógica de perímetro y validación de coordenadas en el Ejercicio 2, así como evitar desbordamientos de pila en el algoritmo DFS.

2. Interacción decisiva

- Consulta (Ejercicio 1 – Streams infinitos): *"Tengo un Stream infinito en Elixir y necesito consumirlo hasta que el conjunto actual sea igual al anterior. ¿Cómo puedo hacerlo sin usar variables mutables?"*

    - Respuesta de la IA: Propuso usar Enum.reduce_while/3, manteniendo el estado anterior en el acumulador y retornando {:halt, resultado} cuando la condición de igualdad se cumpla.

    - Verificación y modificación: Se probó con un grafo pequeño y se descubrió que el acumulador debía guardar también el historial de estados y el paso de estabilización. Se ajustó la propuesta de la IA para que el acumulador fuera una tupla {historial, anterior, paso} y funcionó correctamente.

- Consulta (Ejercicio 2 – Cálculo de perímetro): *"Necesito calcular el perímetro de una figura en una matriz. Tengo la lista de coordenadas {fila, col} de la región, pero no quiero volver a recorrer toda la matriz para ver qué toca el borde o a otro símbolo. ¿Cómo puedo hacerlo eficientemente?"*

    - Respuesta de la IA: Propuso crear un MapSet con las coordenadas de la región. Luego, iterar sobre cada celda y revisar sus 4 vecinos; si un vecino no está en el MapSet, significa que ese lado da hacia afuera o hacia otra región, por lo que suma 1 al perímetro.

    - Verificación y modificación: Se probó el algoritmo con la Región A del ejemplo ({0,0}, {0,1}, {1,0}). El resultado teórico daba 8, y el código de la IA acertó. Se implementó usando Enum.reduce sobre las celdas, obteniendo el mismo resultado exitoso.

    -

3. Enfoques nuevos

- Sugerencia de la IA (Ejercicio 1): En lugar de usar Enum para la validación recursiva (lo cual estaba prohibido por el Requisito 7), sugirió usar MapSet para llevar el control de los centros visitados y de los pares origen-destino ya vistos.
    - Aporte: MapSet es una estructura de datos inmutable que permite búsquedas en tiempo O(1), evitando que la validación de duplicados fuera ineficiente.
    - ¿Se usó?: Sí, en Depuracion y en Grafo.
    - Aprendizaje: Aprendimos que Elixir tiene estructuras nativas (MapSet, Map) que no son Enum ni Stream, y que son perfectas para acumular estado en funciones recursivas puras.
- Sugerencia de la IA (Ejercicio 2): En lugar de crear un string concatenando todo el SVG (lo cual violaría el requisito de no cargar todo en memoria) o usar File.write! en bucle, la IA sugirió usar File.open(ruta, [:write]) para obtener un IO Device y usar IO.puts(archivo, fragmento).
    - Aporte: Esto permitió generar las etiquetas XML <svg>, <rect> y <text> progresivamente, cumpliendo estrictamente con el requisito de escritura por flujo.
    - ¿Se usó?: Sí, en el módulo Svg y en la escritura de reportes.
    - Aprendizaje: Aprendimos que Elixir maneja la I/O de forma muy elegante mediante file descriptors y que podemos hacer streaming de salida sin necesidad de acumular grandes estructuras en memoria.

4. Balance de aprendizaje

- Lo que se comprende sin ayuda: La lectura perezosa (File.stream!), el uso de Streams para filtrado, la creación de Structs con @enforce_keys, el manejo de tuplas, y la escritura de archivos con File.write! y File.stream!.
- Lo que sigue siendo difícil: La implementación de validaciones recursivas complejas sin usar Enum. Aunque se logró en ambos ejercicios, sigue siendo un reto mental adaptar la lógica de "recorrer listas con Enum" a funciones recursivas con acumuladores. También el manejo de punteros en el DFS para no desbordar la pila en mapas o grafos grandes, y entender a fondo cómo funciona la recursión de cola.
- Qué haría diferente: En el próximo trabajo, dedicaría más tiempo al diseño inicial en papel antes de escribir la primera línea de código. También escribiría pruebas unitarias (ExUnit) para verificar que el validador y el parser funcionan correctamente con entradas erróneas antes de integrarlos con el generador de SVG o el buscador DFS.

https://github.com/SSGarcia02/Parcial_2_Programacion3.git